Query execution*

Distributed Data Processing Environments

Lab Guide 5

This session aims at reading the plan that an engine builds for a query, and at relating that plan with the time the query takes. Nothing here needs the cloud, but an instance like the one of Lab 4 does just as well.

Steps First prepare data large enough to be interesting:

1. Work wherever DuckDB[1] is easiest for you: your own machine, the Lab 0 container, or a cloud instance.
2. Start it with `duckdb -ui` if you prefer a browser to a terminal.[2] On a cloud instance, bring the interface to your own browser by connecting with:
   `gcloud compute ssh` *name* `--ssh-flag="-L 4213:localhost:4213"`
3. Generate the standard dataset:

   ```
   INSTALL tpch; LOAD tpch;
   CALL dbgen(sf = 1);
   ```

4. Explore the schema with `SHOW TABLES` and `DESCRIBE ...`
5. Note how many rows each table holds. You will need these numbers later.

Now turn one query into a tree of operators:

1. Write a query that joins at least three tables, filters, and groups. Use your own, or read one of the twenty-two standard ones from the extension itself:

   ```
   SELECT query FROM tpch_queries() WHERE query_nr = 3;
   ```

   On the command line, precede it with `.mode line` to print it readably. The results these queries should produce are in `tpch_answers()`, if you want to check your own.
2. Run it with `EXPLAIN` and draw the tree it prints. Say what each operator does with the rows it receives.
3. For a tree you do not have to draw by hand, ask for the plan as JSON and paste it into a visualizer:[3]

   ```
   EXPLAIN (FORMAT JSON) SELECT ...;
   EXPLAIN (ANALYZE, FORMAT JSON) SELECT ...;
   ```

   The first carries the estimated cardinalities, the second the measured times.
4. Say which operators can work at the same time, and which must wait for another to finish. What does that tell you about where memory is needed?
5. Run the same query with `EXPLAIN ANALYZE`. Now each operator shows how long it took. Where did the time actually go?

*Reviewer mode in the third part. See AI protocol.

[1] `https://duckdb.org/install/`

[2] `https://duckdb.org/docs/stable/core_extensions/ui.html`

[3] `https://db.cs.uni-tuebingen.de/explain/`

6. Match each clause of your query to an operator in the plan. The engine runs them from the bottom up, and not in the order you wrote them: say which clause produced the slowest operator.

Now compare what the engine expected with what it found:

1. `EXPLAIN` prints the rows the engine **expects** each operator to produce, marked with a `~`. `EXPLAIN ANALYZE` prints the rows it **actually** produced. Put the two side by side.
2. Find an operator where the two differ by more than ten times.
3. Explain where the estimate came from, and why the data did not agree with it.
4. Write down your own reading of the plan: which operator dominated, and why. Then ask the agent to criticize it, using the reviewer prompt of the AI protocol. Rule on each criticism against the numbers in the plan, and run whatever extra measurement it proposes.

Finally, find out what makes the engine change its mind:

1. Add a condition that removes almost every row. Predict whether the plan changes, and only then look.
2. Write the joins in a different order in the SQL. Does the plan change? Explain what that says about the language.
3. Generate the data again with a different scale factor. Which operators merely grew, and which were replaced?
4. Read *lineitem* from a Parquet file, and then from a CSV file, instead of from the table. Compare the three scans, and say what each one has to do that the others do not.

Learning Outcomes Describe the purpose of each relational operator in a query plan. Identify factors that impact the choice of query plan. Distinguish estimated from actual cardinality. Attribute the runtime of a query to individual operators.